AI-Assisted Discovery of Weak Randomness in Android Session Tokens

YIFAN YANG, *the university of sydney*

Abstract

```
kotlin.random.FallbackThreadLocalRandom        public final class FallbackThreadLocalRandom extends AbstractPlatformRandom {
kotlin.random.FallbackThreadLocalRandom        @Override // kotlin.random.AbstractPlatformRandom
kotlin.random.FallbackThreadLocalRandom.getIm  public java.util.Random getImpl() {
kotlin.random.FallbackThreadLocalRandom$im...  private final FallbackThreadLocalRandom$implStorage$1 implStorage = new ThreadLocal<java.util.Random>() { //
kotlin.random.FallbackThreadLocalRandom$impl5  public java.util.Random initialValue() {
kotlin.random.FallbackThreadLocalRandom$impl5  return new java.util.Random();
kotlin.random.jdk8.PlatformThreadLocalRandom   package kotlin.random.jdk8;
kotlin.random.jdk8.PlatformThreadLocalRandom   import java.util.Random;
kotlin.random.jdk8.PlatformThreadLocalRandom   import java.util.concurrent.ThreadLocalRandom;
kotlin.random.jdk8.PlatformThreadLocalRandom   import kotlin.random.AbstractPlatformRandom;
kotlin.random.jdk8.PlatformThreadLocalRandom   /* JADX INFO: compiled from: PlatformThreadLocalRandom.kt */
kotlin.random.jdk8.PlatformThreadLocalRandom   @Metadata(d1 = {"\u0000*\n\u0002\u0018\u0002\n\u0002\n\u0018\u0002\n\u0002\b\u0002\n\u0002\u0018\u0002\n\u0002
kotlin.random.jdk8.PlatformThreadLocalRandom   public final class PlatformThreadLocalRandom extends AbstractPlatformRandom {
kotlin.random.jdk8.PlatformThreadLocalRandom   @Override // kotlin.random.AbstractPlatformRandom
kotlin.random.jdk8.PlatformThreadLocalRandom   @Override // kotlin.random.Random
kotlin.random.jdk8.PlatformThreadLocalRandom   @Override // kotlin.random.Random
kotlin.random.jdk8.PlatformThreadLocalRandom   @Override // kotlin.random.Random
kotlin.random.jdk8.PlatformThreadLocalRandom   @Override // kotlin.random.Random
kotlin.random.jdk8.PlatformThreadLocalRandom.  public Random getImpl() {
kotlin.random.jdk8.PlatformThreadLocalRandom.  ThreadLocalRandom threadLocalRandomCurrent = ThreadLocalRandom.current();
kotlin.random.jdk8.PlatformThreadLocalRandom.  Intrinsics.checkNotNullExpressionValue(threadLocalRandomCurrent, "current()");
kotlin.random.jdk8.PlatformThreadLocalRandom.  return threadLocalRandomCurrent;
kotlin.random.jdk8.PlatformThreadLocalRandom.  return ThreadLocalRandom.current().nextDouble(until);
kotlin.random.jdk8.PlatformThreadLocalRandom.  return ThreadLocalRandom.current().nextInt(from, until);
kotlin.random.jdk8.PlatformThreadLocalRandom.  return ThreadLocalRandom.current().nextLong(until);
kotlin.random.jdk8.PlatformThreadLocalRandom.  return ThreadLocalRandom.current().nextLong(from, until);
```


1. Introduction

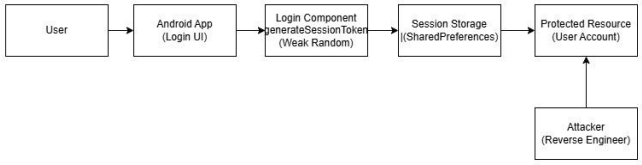
Android applications are commonly distributed as APK files, which can be reverse engineered to inspect their internal logic. Security analysis of APKs is important because developers may incorrectly implement cryptographic or randomness-related functionality.

In this report, we analyse a provided APK and identify a vulnerability related to insecure randomness in session token generation. The vulnerability enables attackers to predict authentication tokens and potentially impersonate users.

2. System and Threat Model

2.1. System Overview

The application provides user authentication functionality through a login interface. After successful login, a session token is generated and stored locally using SharedPreferences. This token is used to maintain the user's authenticated session and control access to protected resources.



2.2. Assets

The key assets include: Session tokens (sessionToken), User authentication state , User account data2. resources.

2.3. Attacker Model

We assume an attacker with the following capabilities:

1) Ability to reverse engineer the APK

2) Ability to interact with the application (e.g., login/register)

3) Ability to observe or collect session tokens

2.4. Attacker Goals

The attacker aims to:

1) Predict or reproduce valid session tokens

2) Bypass authentication mechanisms

3) Impersonate legitimate users

3. Methodology

The APK was decompiled using JADX GUI to recover readable Java source code. Static analysis was performed by searching for keywords such as Random, token, and generate.

The session token generation logic was identified in the Login class. AI assistance was used to guide the identification of insecure randomness patterns and validate the security implications.

4. Vulnerability Analysis

4.1. Code Location

Class: Login

Method: generateSessionToken()

4.2. Code Snippet

```
Random random = new Random();
StringBuilder sb = new StringBuilder(16);
for (int i = 0; i < 16; i++) {
  sb.append("ABCDEFGHIJKLMNOPQRSTUVWXYZa-
bcdefghijklmnopqrstuvwxyz0123456789"
      .charAt(random.nextInt(62)));
}
return sb.toString();
```

4.3. Issue Description

The application uses java.util.Random to generate session tokens. This is not a cryptographically secure random number generator. The output of Random is deterministic and depends on an internal seed, which is typically based on system time.

As a result, the generated session tokens may be predictable or reproducible by an attacker.

4.4. Security Impact

Since session tokens are used for authentication, predictable tokens introduce serious risks:

1) Session hijacking

2) Authentication bypass

3) Unauthorized access to user accounts

5. Attack Scenario

A realistic attack can be performed as follows:

1)The attacker decompiles the APK to analyse token generation logic.

2)The attacker identifies the use of java.util.Random in session token generation.

3)The attacker logs into their own account and collects generated session tokens.

4)Using knowledge of the algorithm and timing information, the attacker approximates the random seed.

5)The attacker predicts future or concurrent session tokens.

6)The attacker uses a predicted token to impersonate another user and gain unauthorized access.

6. Mitigation

The vulnerability can be mitigated by replacing java.util.Random with a cryptographically secure alternative.

6.1. Secure Implementation

```
SecureRandom random = new SecureRandom();
```

6.2. Rationale

SecureRandom is designed for security-sensitive use cases and produces unpredictable values that cannot be feasibly reproduced by attackers. This prevents token prediction and ensures stronger authentication security.

7. Conclusion

This report demonstrates how improper use of randomness can introduce critical security vulnerabilities in Android applications. By using a non-cryptographic random number generator for session tokens, the application exposes itself to token prediction attacks.

Replacing insecure randomness with secure alternatives such as SecureRandom is essential to protect authentication mechanisms and user data.